

Introduction to Hyperparameter Optimisation

Hyperparameter optimisation lab

Nick Kocher

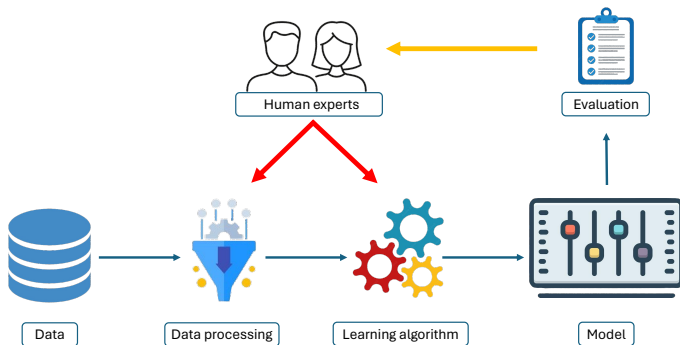
Hadar Shavit

Holger Hoos

8th Apr, 2026



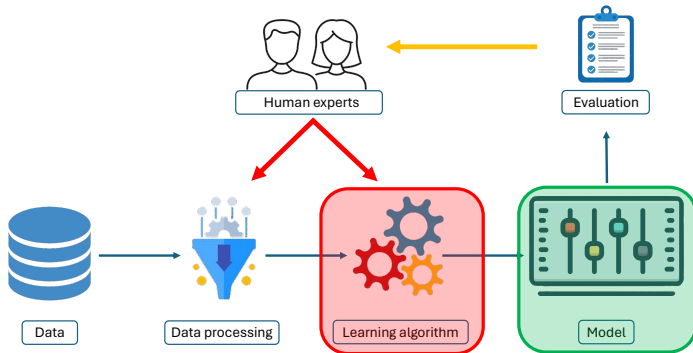
Motivation



A typical ML system.

- ▶ There are many design choices that require human intervention (*i.e.*, are selected manually via *trial and error*).
- ▶ Performance of the ML system is highly dependent on these choices.

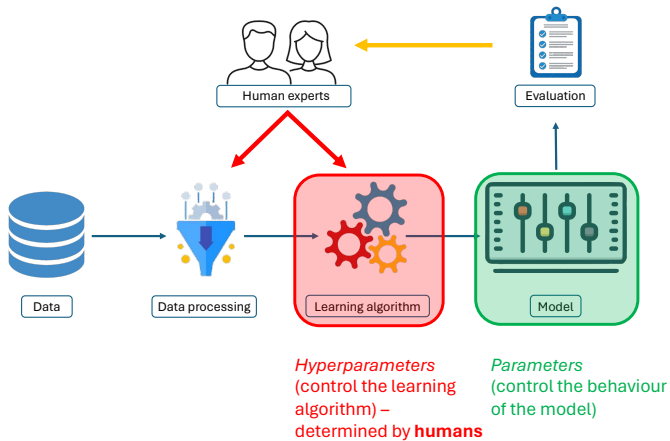
Motivation



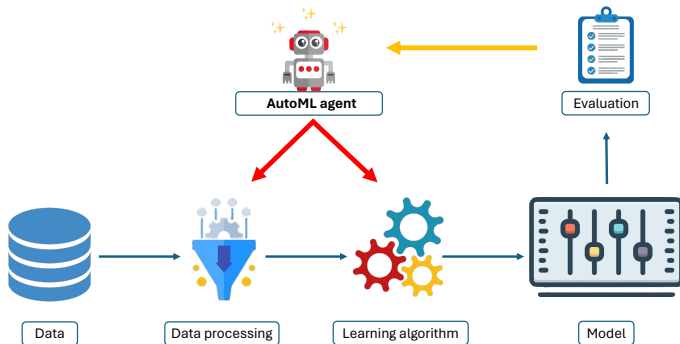
A typical ML system.

- There is a difference between a *learning algorithm* and a *model*!

Motivation



Motivation



- ▶ Automated Machine Learning: automating tasks which are executed by human experts
- ▶ Why? Finding good settings requires expert domain knowledge; it is a tedious and expensive process; it is suboptimal (machines can do better!)

Organisation

- ▶ three meetings, two assignments, presentation at the end
- ▶ Groups of two students
- ▶ Check guidelines on Moodle
- ▶ Check correct registration in RWTH online
- ▶ Unofficial offer: You can deregister up until the second assignment (by sending us an email)
- ▶ Feedback and questions are highly appreciated (email, appointment or after the meetings)
- ▶ Office hours by appointment at Wednesday 14:30

Assignments

- ▶ Two assignments
- ▶ First assignment: introduction to HPO
- ▶ The Second assignment lets you explore your interest in state-of-the-art methods (and present them)
- ▶ Report & code required for both assignments, presentation for the second assignment
- ▶ No extensions (unless sickness or extreme cases)
- ▶ AI is only allowed for polishing reports and needs to be marked. All other detected cases result in failures.

Attendance

- ▶ All meetings are mandatory
- ▶ If you cannot attend a meeting, let us know (needs to be justified)
- ▶ 3 minutes break to register attendance on Moodle

Today

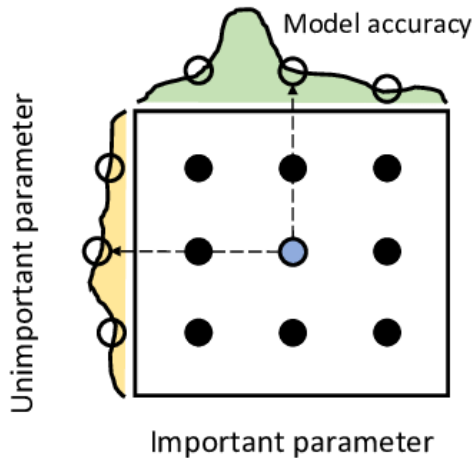
- ▶ Basics of HPO
- ▶ Group building
- ▶ Understanding and visualisation of HPO
- ▶ 1 hour to start working and ask questions

Hyperparameter optimisation

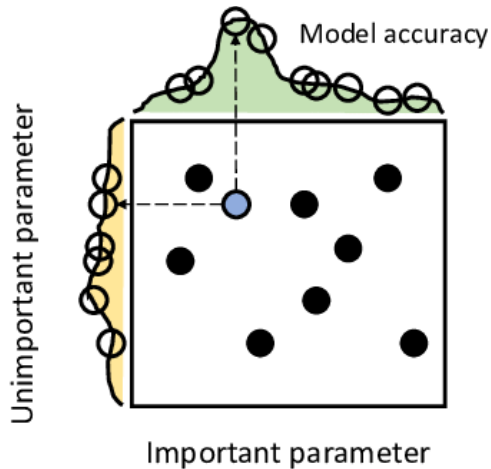
- ▶ The problem of finding good settings of the learning algorithm's hyperparameters such that the performance of the model is maximised
- ▶ A (hyperparameter) configuration = a set of instantiated values for each hyperparameter of a learning algorithm
- ▶ Each configuration is mapped to model's performance (according to the metric of choice)
- ▶ The goal is to find a configuration which yields the best-performing model

```
class sklearn.ensemble.RandomForestClassifier(n_estimators=100, *, criterion='gini', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='sqrt', max_leaf_nodes=None, min_impurity_decrease=0.0, bootstrap=True, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False, class_weight=None, ccp_alpha=0.0, max_samples=None, monotonic_cst=None)
```

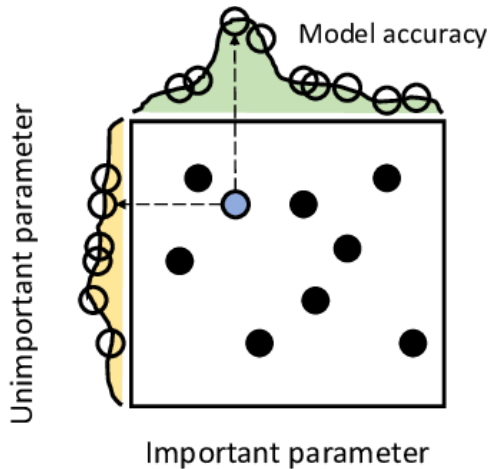
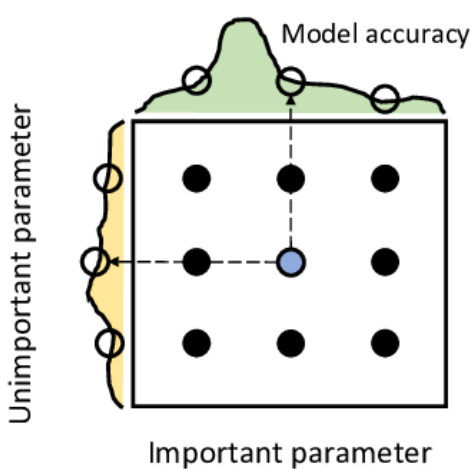
Grid search



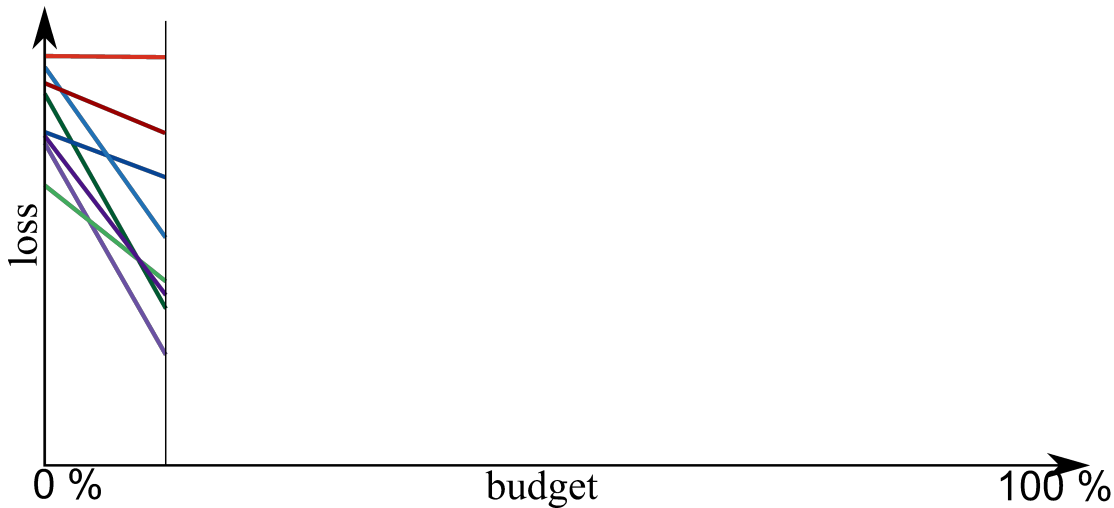
Random search



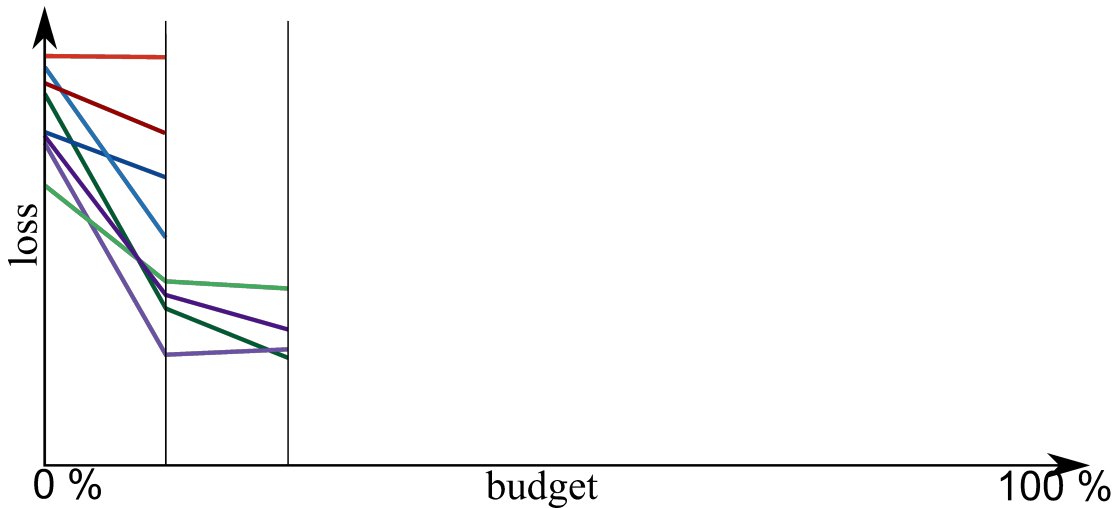
Grid vs Random search



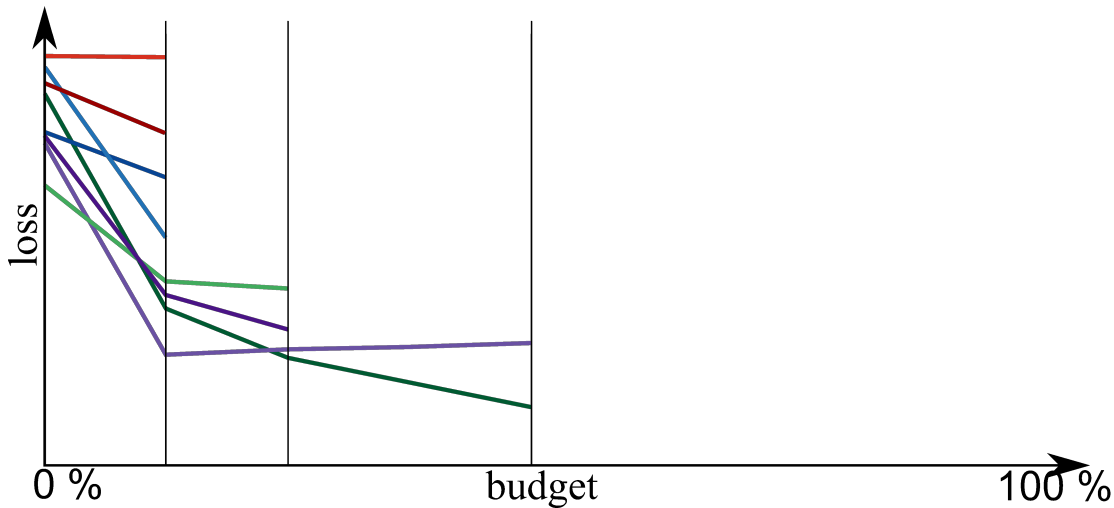
Successive halving



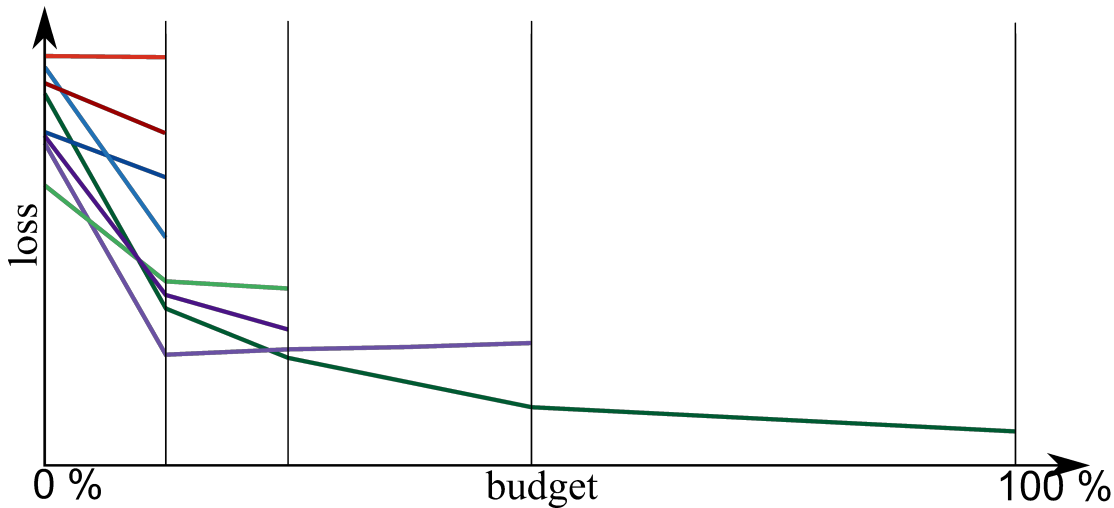
Successive halving



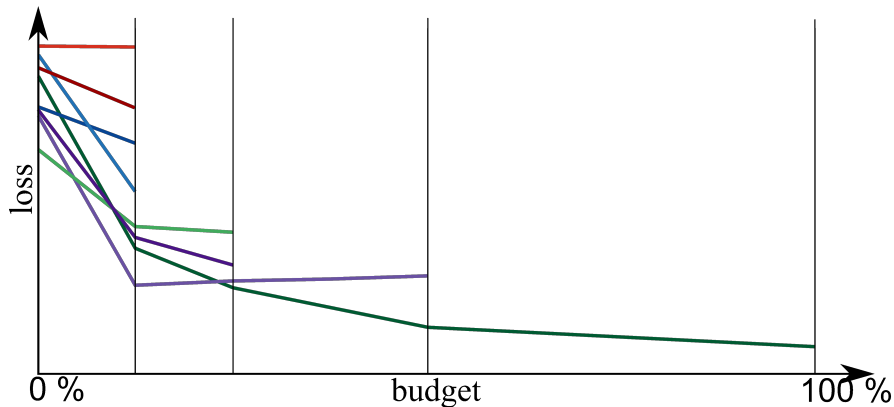
Successive halving



Successive halving

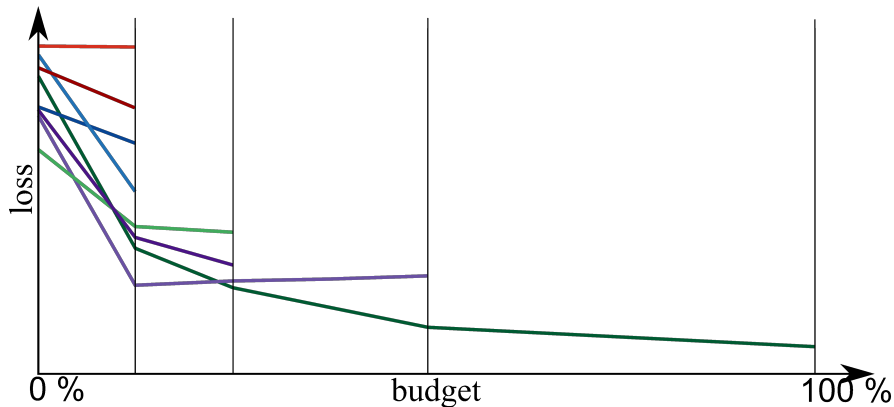


Successive halving



In case of a lot of bad configurations, can you think of a modification of the search?

Successive halving



Expansion: keep $\frac{1}{\eta}$ configurations. Special Case: Original Successive halving with $\eta = 2$

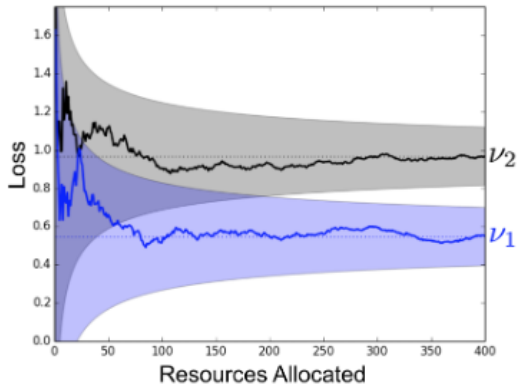
Summary: Successive halving

Random search is expensive. Most configurations are uninteresting and yield poor performance.

- ▶ Assumption: Bad configurations behave bad early in the training
- ▶ Iteratively increase budget and reduce number of configurations → Race
- ▶ Budget: training data, cross validation folds, number of trees, training time
- ▶ First assignment use cross validation folds. You are encouraged to try out different budgets and play around with them

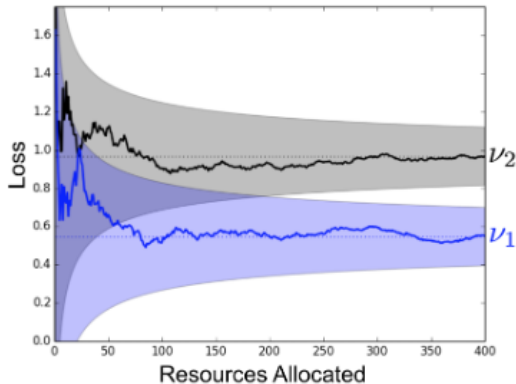
Hyperband

- Problem with successive halving:



Hyperband

- Problem with successive halving:



- Solution: Run successive halving multiple times for different trade-offs between number of configurations and initial budget.

Hyperband

i	$s = 4$	
	n_i	r_i
0	81	1
1	27	3
2	9	9
3	3	27
4	1	81

Hyperband

i	$s = 4$		$s = 3$	
	n_i	r_i	n_i	r_i
0	81	1	27	3
1	27	3	9	9
2	9	9	3	27
3	3	27	1	81
4	1	81		

Hyperband

i	$s = 4$		$s = 3$		$s = 2$		$s = 1$		$s = 0$	
	n_i	r_i	n_i	r_i	n_i	r_i	n_i	r_i	n_i	r_i
0	81	1	27	3	9	9	6	27	5	81
1	27	3	9	9	3	27	2	81		
2	9	9	3	27	1	81				
3	3	27	1	81						
4	1	81								

How to randomly sample configurations

- ▶ Use ConfigSpace library
- ▶ Not allowed to use in the first assignment (allowed in the second)

How to randomly sample configurations

- ▶ Use ConfigSpace library
- ▶ Create hyperparameters for algorithms very elegantly

```
>>> from ConfigSpace import ConfigurationSpace
>>>
>>> cs = ConfigurationSpace({
...     "myfloat": (0.1, 1.5),           # Uniform Float
...     "myint": (2, 10),               # Uniform Integer
...     "species": ["mouse", "cat", "dog"], # Categorical
... })
>>> configs = cs.sample_configuration(2)
```

ConfigSpace Base Documentation

- ▶ Not allowed to use in the first assignment (allowed in the second)

How to randomly sample configurations

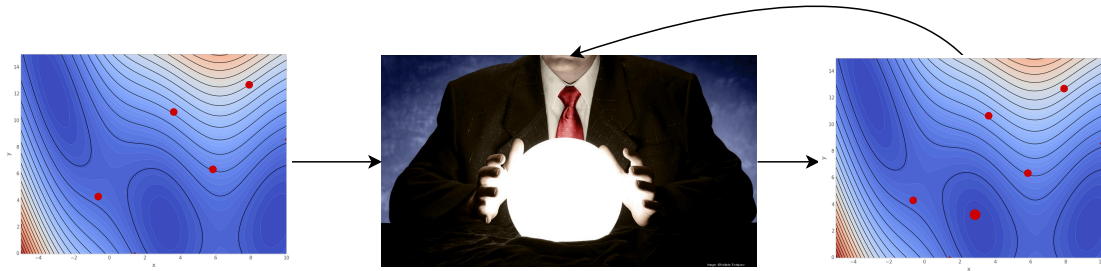
- ▶ Use ConfigSpace library
- ▶ Create hyperparameters for algorithms very elegantly

```
>>> from ConfigSpace import ConfigurationSpace
>>>
>>> cs = ConfigurationSpace({
...     "myfloat": (0.1, 1.5),           # Uniform Float
...     "myint": (2, 10),               # Uniform Integer
...     "species": ["mouse", "cat", "dog"], # Categorical
... })
>>> configs = cs.sample_configuration(2)
```

ConfigSpace Base Documentation

- ▶ Initialise an algorithm using SVM(**config)
- ▶ Not allowed to use in the first assignment (allowed in the second)

Improving upon random search



Bayesian optimisation

1. Collect random samples

Bayesian optimisation

1. Collect random samples
2. Create a *surrogate model*: a machine learning/statistical model:

$$X \rightarrow P \times V$$

X – inputs, P – performances, V – performance variance

Bayesian optimisation

1. Collect random samples
2. Create a *surrogate model*: a machine learning/statistical model:

$$X \rightarrow P \times V$$

X – inputs, P – performances, V – performance variance

3. Use an *acquisition function* – tells us where to search, using the surrogate

Bayesian optimisation

1. Collect random samples
2. Create a *surrogate model*: a machine learning/statistical model:

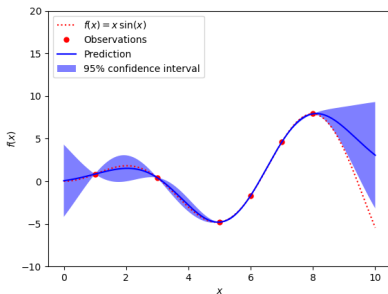
$$X \rightarrow P \times V$$

X – inputs, P – performances, V – performance variance

3. Use an *acquisition function* – tells us where to search, using the surrogate
4. Sample additional point(s)
5. Go back to 2 unless the budget is exhausted

Surrogate model

- ▶ Traditionally, Gaussian processes:
Statistical method that models the data as a multivariate normal distribution of functions
 - ↪ Not good with discrete inputs (integer and categorical hyperparameters)
 - ↪ Doesn't scale well



Surrogate model

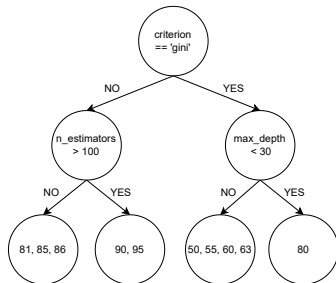
Random forest:

- ▶ Variance calculated based on predictions of the trees:
 - ▶ Each tree predicts a value and computes

Surrogate model

Random forest:

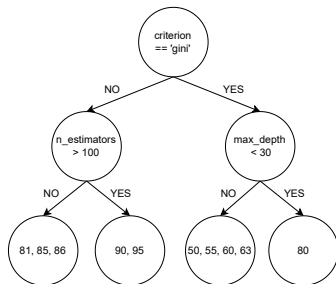
- ▶ Variance calculated based on predictions of the trees:
 - ▶ Each tree predicts a value and computes
 - ▶ Calculating the variance of the different trees



Surrogate model

Random forest:

- ▶ Variance calculated based on predictions of the trees:
 - ▶ Each tree predicts a value and computes
 - ▶ Calculating the variance of the different trees



- ▶ Every other ML model that can return mean and variance

Acquisition functions

Improvement:

$$I(x) = \max(f(x) - f(x^*), 0) \quad (1)$$

Acquisition functions

Improvement:

$$I(x) = \max(f(x) - f(x^*), 0) \quad (1)$$

► Probability improvement:

$$PI(x) = P(I(x) > 0) = P(f(x) > f(x^*)) \quad (2)$$

Acquisition functions

Improvement:

$$I(x) = \max(f(x) - f(x^*), 0) \quad (1)$$

► Probability improvement:

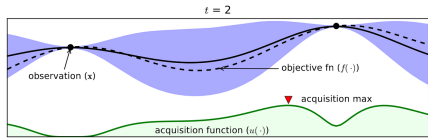
$$PI(x) = P(I(x) > 0) = P(f(x) > f(x^*)) \quad (2)$$

► Expected improvement:

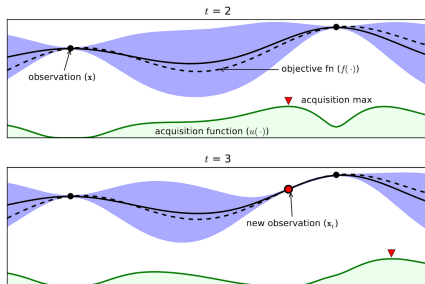
$$EI(x) = E[I(x)] \quad (3)$$

For full calculation look at the AutoML book on page 9

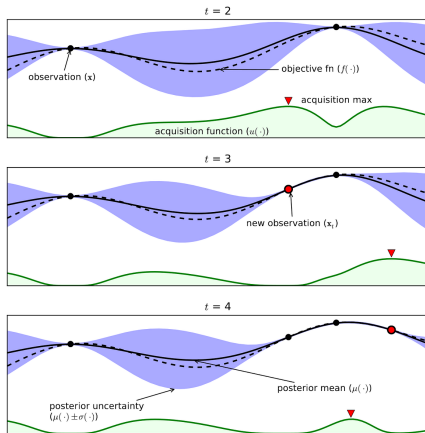
Bayesian optimisation



Bayesian optimisation

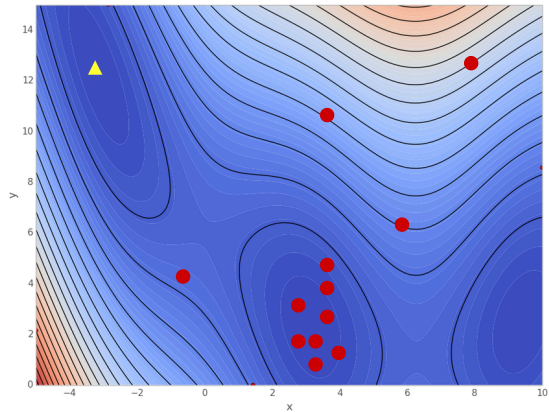


Bayesian optimisation



Exploration vs Exploitation

Exploration vs exploitation



Exploration vs exploitation: multi-armed bandits



Exploration vs Exploitation

- ▶ “Pretend” that we have a higher best value will guide EI to give more explorative candidates

Exploration vs Exploitation

- ▶ “Pretend” that we have a higher best value will guide EI to give more explorative candidates
- ▶ Random sampling every few iterations

Hierarchical configuration spaces

Hierarchical configuration spaces

- ▶ In some cases certain hyperparameters are active depending on the value of another hyperparameter

Hierarchical configuration spaces

- ▶ In some cases certain hyperparameters are active depending on the value of another hyperparameter
- ▶ Example: multi-layer perceptron (MLP)
 - ▶ # of layers
 - ▶ # of neurons in 1st layer
 - ▶ # of neurons in 2nd layer (active only if there are 2 or more layers)
 - ▶ # of neurons in 3rd layer (active only if there are 3 or more layers AND layer 2 has at least 16 neurons)
- ▶ Handling with a surrogate model: imputing a value

Hierarchical configuration spaces

How to process hierarchical configuration spaces?

Hierarchical configuration spaces

How to process hierarchical configuration spaces?

Hierarchical configuration spaces

How to process hierarchical configuration spaces?

Experimental setup & Visualisation

Experimental setup

- ▶ How many times to run an algorithm?

Experimental setup

- ▶ How many times to run an algorithm?
 $\leadsto$ As many as possible, 3 is the bare minimum, 5 is preferable
- ▶ How many benchmarks?
- ▶ Which baselines?

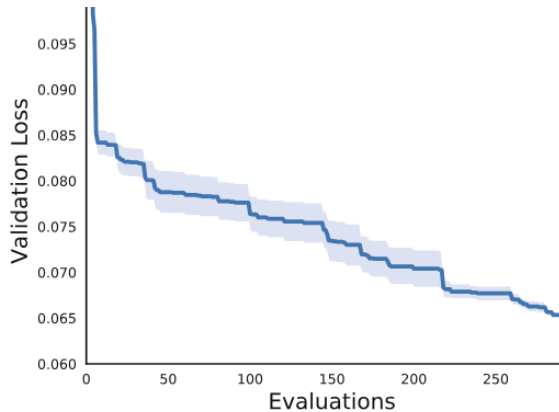
Experimental setup

- ▶ How many times to run an algorithm?
 - ↪ As many as possible, 3 is the bare minimum, 5 is preferable
- ▶ How many benchmarks?
- ▶ Which baselines?
 - ↪ No specific number, depending on the message we want to convey
- ▶ What budget?

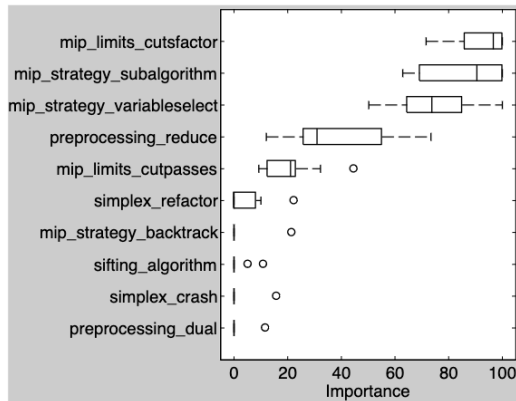
Experimental setup

- ▶ How many times to run an algorithm?
 - ↪ As many as possible, 3 is the bare minimum, 5 is preferable
- ▶ How many benchmarks?
- ▶ Which baselines?
 - ↪ No specific number, depending on the message we want to convey
- ▶ What budget?
 - ↪ In HPO, running time is very important, sometimes a number of evaluations. Budget must be equal for all methods

Visualization

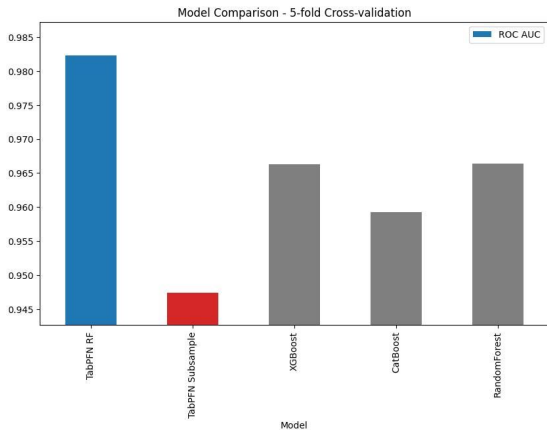


Convergence Plot

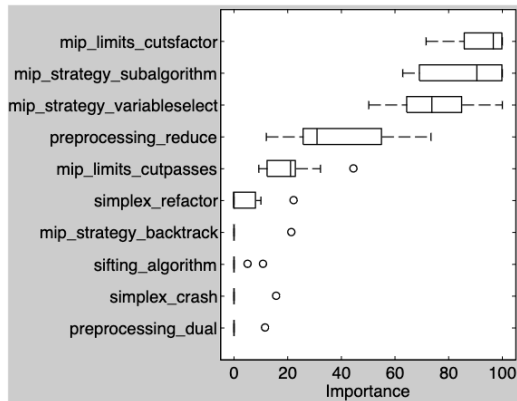


Box Plot

Bar vs Box plot



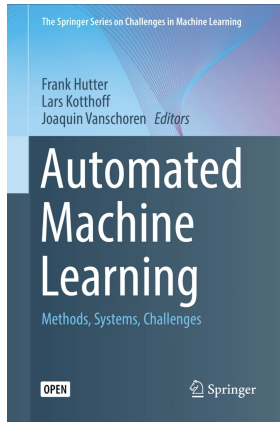
Bar plot



Box Plot

Further Material

- ▶ First book on AutoML: [AutoML: Methods, Systems, Challenges](#)
- ▶ [MOOC on AutoML](#)
- ▶ "Further Material" section with a list of papers



Building Groups...

and a 10 minute break